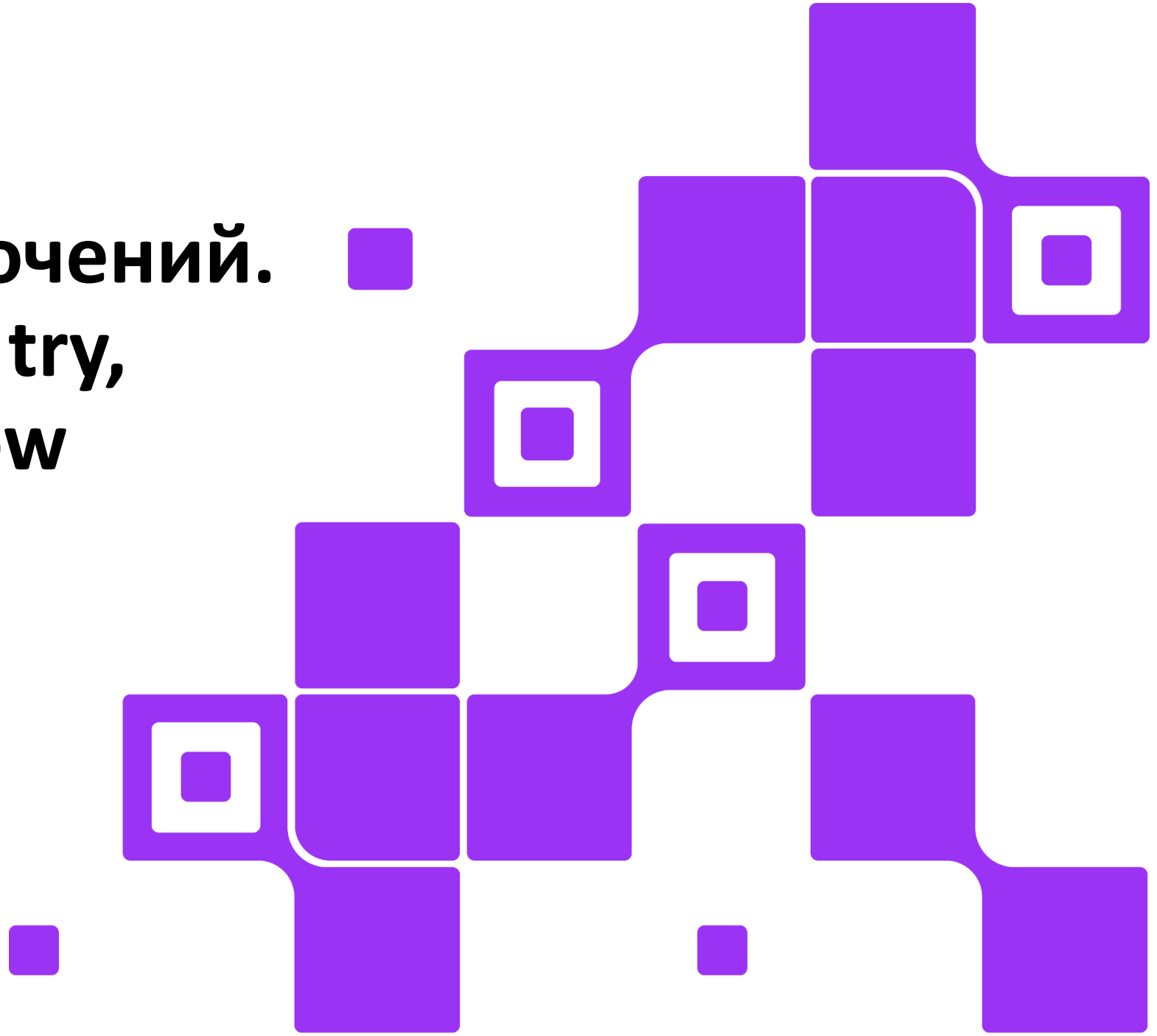
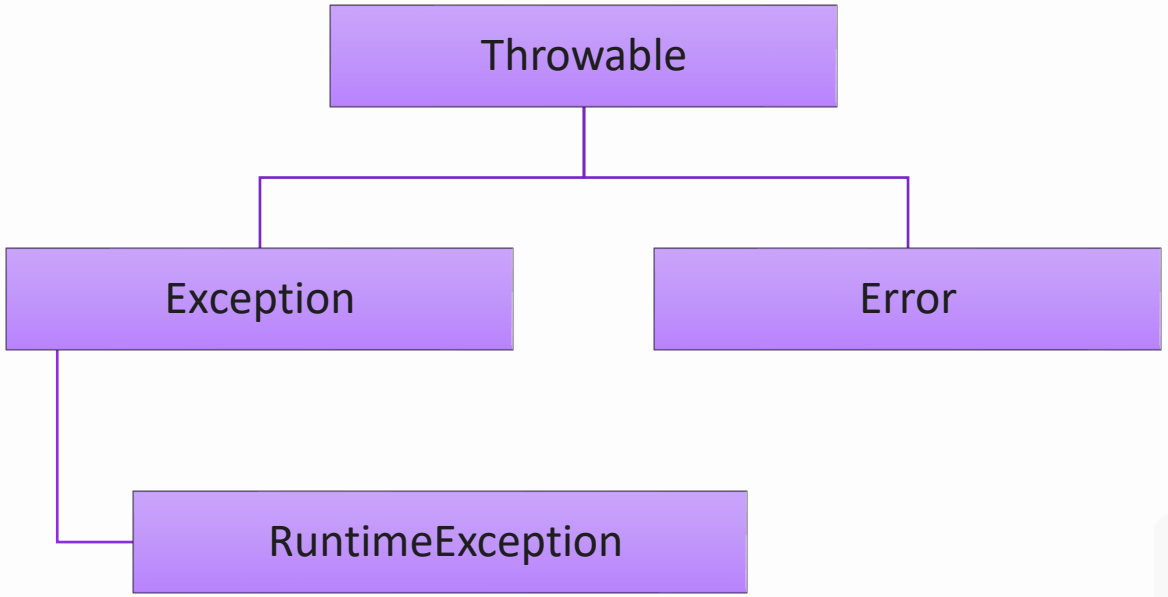
Обработка исключений. Ключевые слова try, catch, finally, throw



- **исключение** Java представляет собой объект, описывающий исключительное (т.е. ошибочное) состояние, которое произошло внутри фрагмента кода
- при возникновении исключительного состояния в методе, вызвавшем ошибку, генерируется объект, представляющий это исключение
- метод может обработать исключение самостоятельно или передать его дальше.
- исключения могут быть сгенерированы исполняющей средой Java или вручную в коде



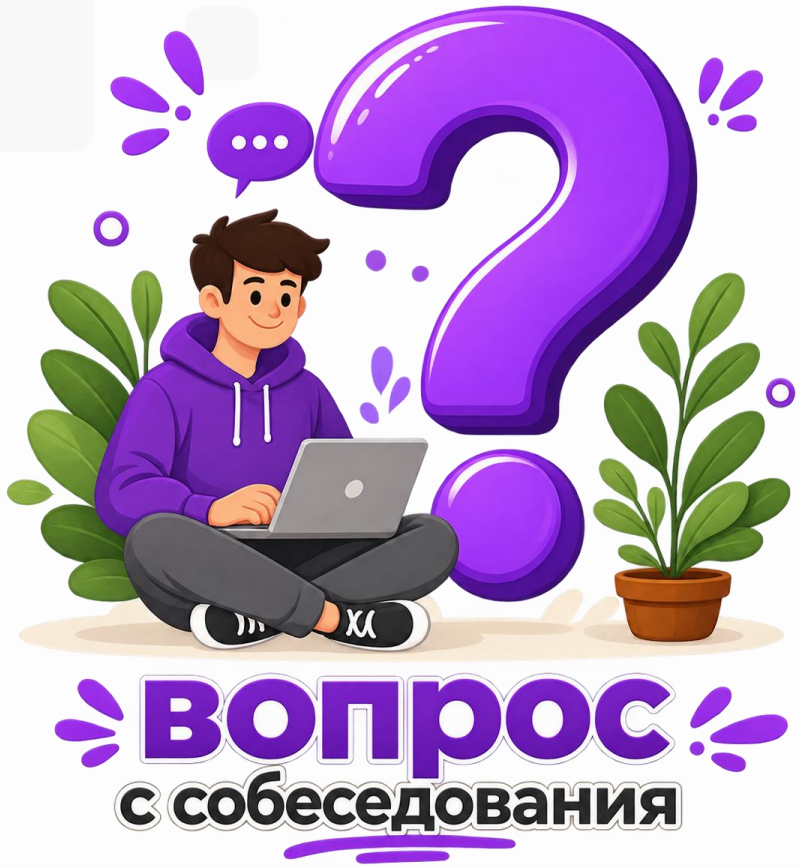
- все типы исключений являются подклассами встроенного класса Throwable
- Exception используется для представления исключительных условий, которые должны перехватываться пользовательскими программами (деление на ноль, недопустимое индексирование массивов)
- Error определяет исключения, которые не должны перехватываться программой в обычных условиях (переполнение стека)

- В блоке **try** код выполняется как обычно, и если исключение не возникает, выполнение продолжается за блоком try.
- Ключевое слово **catch** используется для определения блока кода, который будет выполнен в случае, если возникнет исключение.
- Ключевое слово **finally** используется для определения блока кода, который будет выполнен в любом случае, независимо от того, возникло исключение или нет. Блок finally может содержать код для очистки ресурсов, например, закрытие файлов или сетевых соединений.

```
try {
    // блок кода, где отслеживаются ошибки
}
catch (ТипИсключения1 объектИсключения) {
    // обработка исключений для ТипИсключения1
}
catch (ТипИсключения2 объектИсключения) {
    // обработка исключений для ТипИсключения2
}

…
finally {
    // блок кода, подлежащий выполнению
    // после окончания блока try
}
```

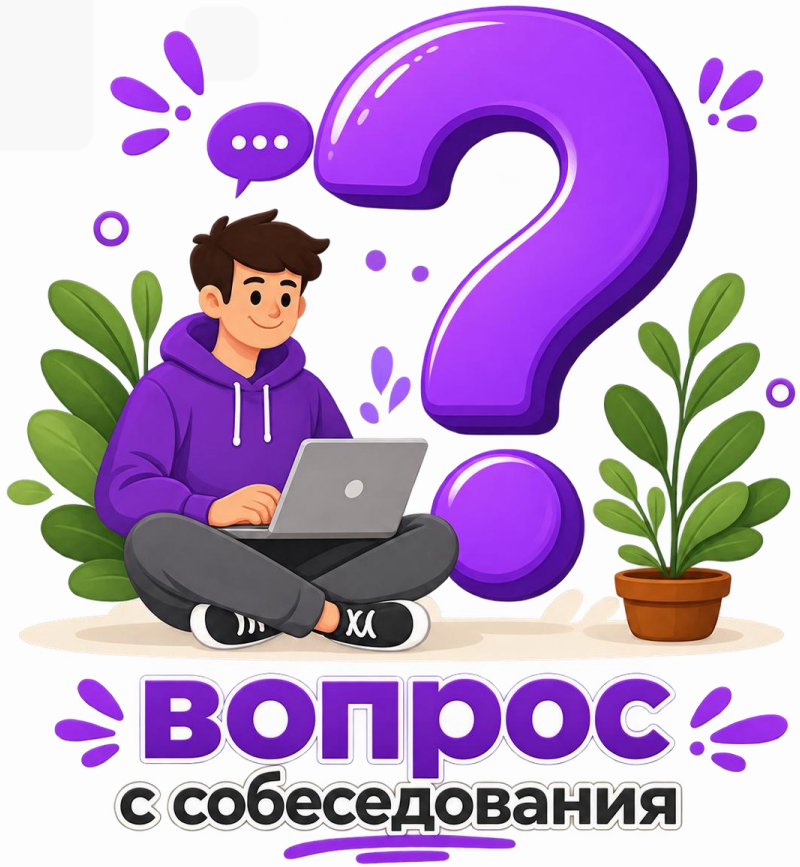
IThub



Возможно ли использование блока try-finally (без catch)?

Такая запись допустима, но смысла в такой записи не так много, всё же лучше иметь блок catch, в котором будет обрабатываться необходимое исключение.

IThub

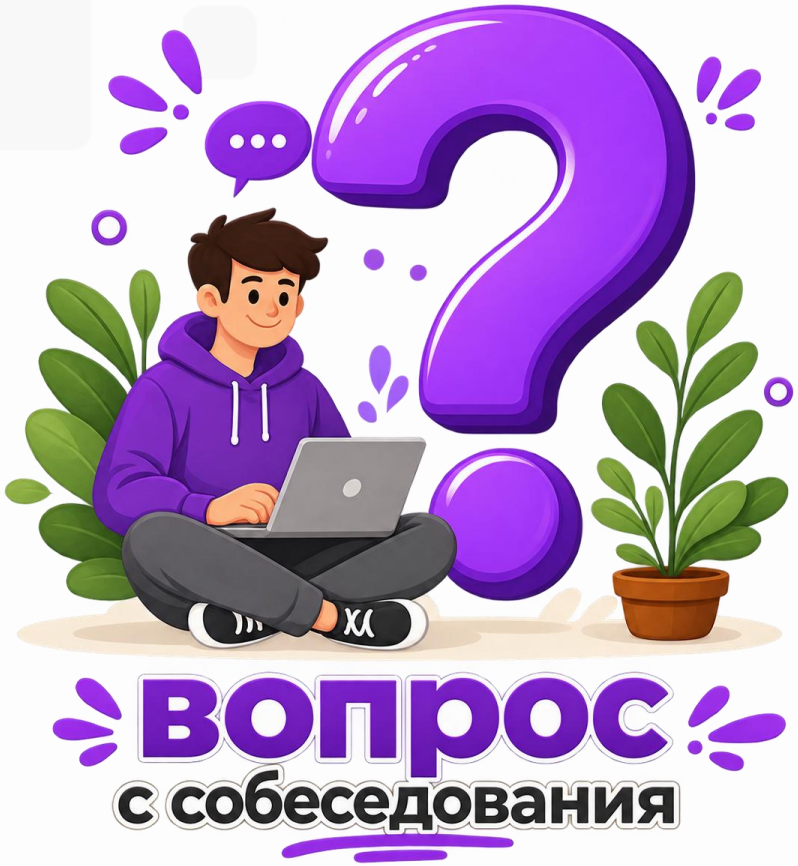


Может ли один блок catch отлавливать сразу несколько исключений?

В Java 7 стала доступна новая языковая конструкция, с помощью которой можно перехватывать несколько исключений одним блоком catch:

```
try {
   //...
} catch(IOException | SQLException ex) {
   //...
}
```

IThub



Что представляет из себя ошибки класса Error?

Ошибки класса Error представляют собой наиболее серьёзные проблемы уровня JVM. Например, исключения такого рода возникают, если закончилась память доступная виртуальной машине. Обрабатывать такие ошибки не запрещается, но делать этого не рекомендуется.

```java
public class ExceptionsExample {

    public void notCaughtException() {
        int d = 0;
        int k = 5;

        int a = k / d;
    }
```

```java
@Test
public void notCaughtExceptionTest() {
    exceptionsExample.notCaughtException();
}
```



```java
public void caughtException() {
    int d = 0;
    int k = 5;

    try {
        int a = k / d;
        System.out.println("it is not output");
    } catch (ArithmeticException e) {
        System.out.println("Arithmetic Exception");

        System.out.println("Message: " + e.getMessage());
        e.printStackTrace();
    }

    System.out.println("After catch");
}
```

```java
@Test
public void notCaughtExceptionTest() {
    exceptionsExample.caughtException();
}
```



Оператор throw

```
throw ThrowableInstance;
```

- используется для явной генерации исключений
- ThrowableInstance должен быть объектом типа Throwable или подклассом Throwable
- поток выполнения останавливается сразу после оператора throw

Конструкция finally

- позволяет создать блок кода, который будет выполняться после завершения блока try/catch и перед кодом, следующим после try/catch
- выполняется независимо от того, сгенерировано исключение или нет

Задача

IThub

Напишите программу, которая запрашивает у пользователя возраст.

Программа должна:

- Считать возраст через Scanner.
- В блоке try проверить возраст.
- Если пользователь ввёл отрицательное число — самостоятельно выбросить исключение с помощью throw new IllegalArgumentException.
- Если пользователь ввёл не число, например "abc", обработать это через catch.
- Если возраст корректный — вывести:

  Ваш возраст: 25
- В блоке finally всегда вывести:

  Проверка завершена

```
public static void main(String[] args) {

    Scanner scanner = new Scanner(System.in);

    try {
        ...
        if (age < 0) {
          throw new IllegalArgumentException(
                "Возраст не может быть отрицательным");
         }
         ...
    } catch (InputMismatchException e) {
            System.out.println("Ошибка: необходимо ввести число");
    }  catch (IllegalArgumentException e) {
           System.out.println(e.getMessage()); // здесь не цифры

    } finally {
       // «Проверка завершена»
    }
}
```

Конструкция throws

```
тип имя-метода(список-параметров) throws список-исключений {
    // тело метода
}
```

- метод сообщает о поведении, способному приводить к исключениям
- к объявлению метода добавляется конструкция throws, где перечисляются типы исключений, которые может генерировать метод
- для всех исключений, кроме исключений типа Error, RuntimeException или любых их подклассов

IThub

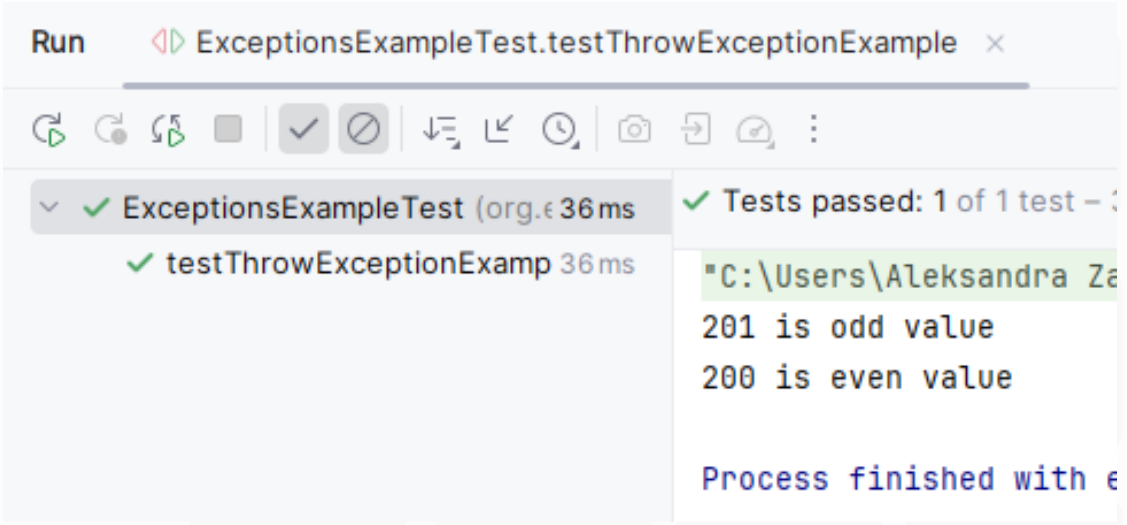
```java
public void throwExceptionExample(int value) throws Exception{
    int mod = value % 2;
    if (mod == 0) {
        System.out.println(value + " is even value");
    } else {
        String message = value + " is odd value";
        System.out.println(message);

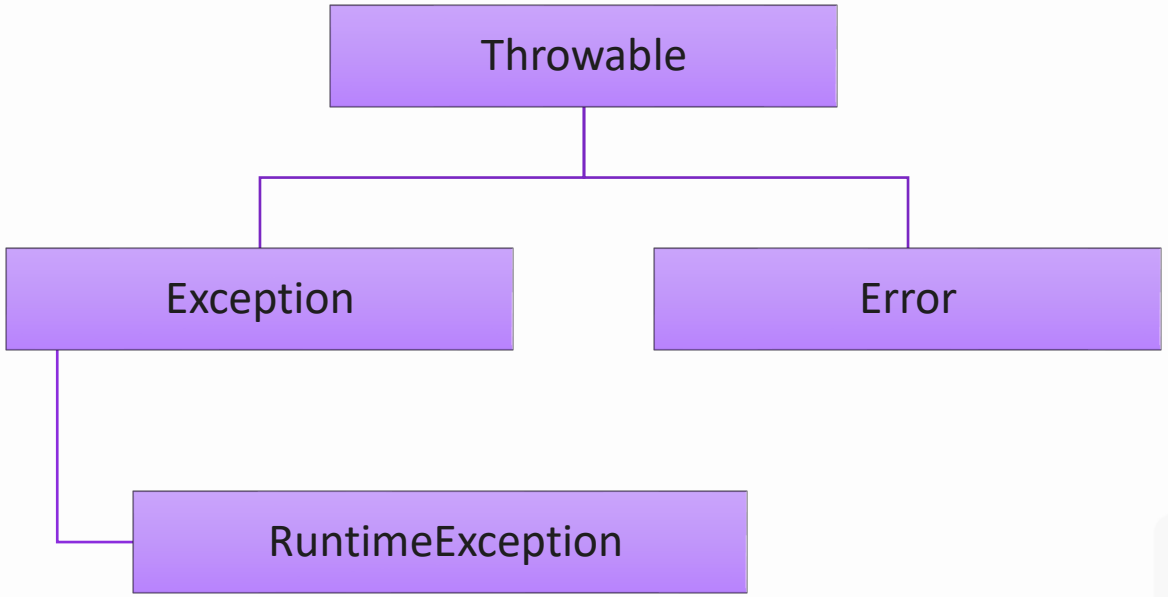
        throw new Exception("message");
    }
}
```

```java
@Test
public void testThrowExceptionExample() {

    Assertions.assertThrows(Exception.class, () -> {
        exceptionsExample.throwExceptionExample(201);
    });

    Assertions.assertDoesNotThrow(() -> {
        exceptionsExample.throwExceptionExample(200);
    });
}
```

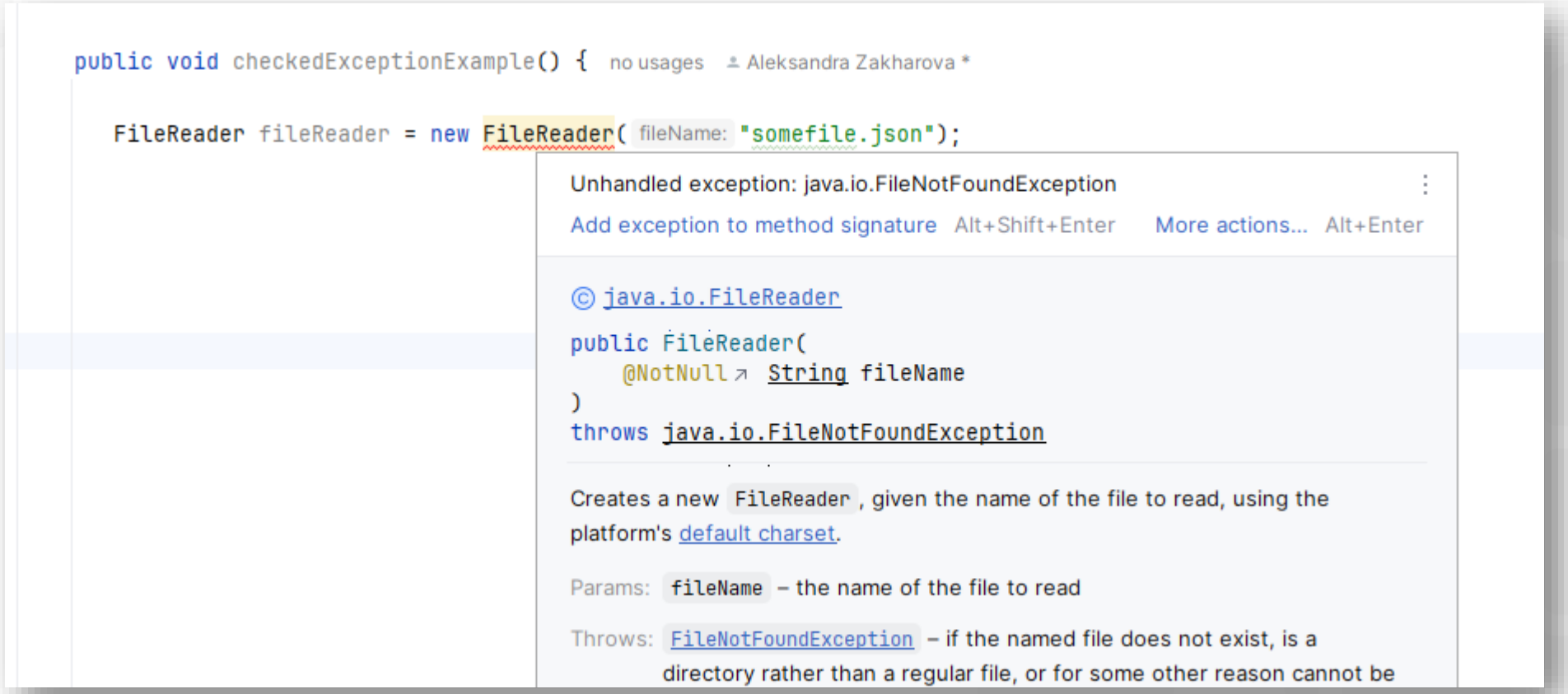




- Exception — базовый класс исключений; его наследники могут быть **проверяемыми** (checked), которые нужно обработать через try-catch или throws.
- RuntimeException — наследник Exception; это **непроверяемые** (unchecked) исключения, компилятор не требует их обрабатывать.

Проверяемые и непроверяемые исключения

1. **Проверяемые** (**checked**) **исключения** (проверяются во время компиляции)



IThub

1 решение

```
public void checkedExceptionExample() throws FileNotFoundException {

  FileReader fileReader = new FileReader("somefile.json");

```

2 решение

```
public void checkedExceptionExample() {

   try {
      FileReader fileReader = new FileReader("somefile.json");
   } catch (FileNotFoundException exception) {
      System.out.println("ERROR: " + exception.getMessage());
   } finally {
      System.out.println("finally");
   }
}
```

2. **Непроверяемые (unchecked) исключения** (не вызывают ошибки во время компиляции)

```
public void uncheckedExceptionExample(int value) {
    int[] numbers = new int[10];

    numbers[value] = value;
}
```

`ArrayIndexOutOfBoundsException extends IndexOutOfBoundsException extends RuntimeException`

```
@Test
public void testUncheckedExceptionExample() {
    Assertions.assertDoesNotThrow(() -> {
        exceptionsExample.uncheckedExceptionExample(1);
    });

    Assertions.assertThrows(ArrayIndexOutOfBoundsException.class, () -> {
        exceptionsExample.uncheckedExceptionExample(200);
    });
}
```

```
java.lang.ArrayIndexOutOfBoundsException: Index 200 out of bounds for length 10

    at org.example.exceptions.ExceptionsExample.uncheckedExceptionExample(ExceptionsEx
    at org.example.exceptions.ExceptionsExampleTest.testUncheckedExceptionExample(Exce
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
```

IThub

Создание собственных подклассов Exception

IThub

```
public class MyException extends Exception {
    private int detail;

    public MyException(int detail) {
        this.detail = detail;
    }

    public MyException() {
    }

    @Override
    public String toString() {
        return "MyException [" +
                "detail=" + detail +
                ']';
    }
}
```

```
public void myExceptionExample(int a) throws MyException {
    System.out.println("\nMy Exception example");

    if (a > 100) {
        throw new MyException(a);
    } else {
        System.out.println("Standard value: " + a);
    }
}
```

IThub

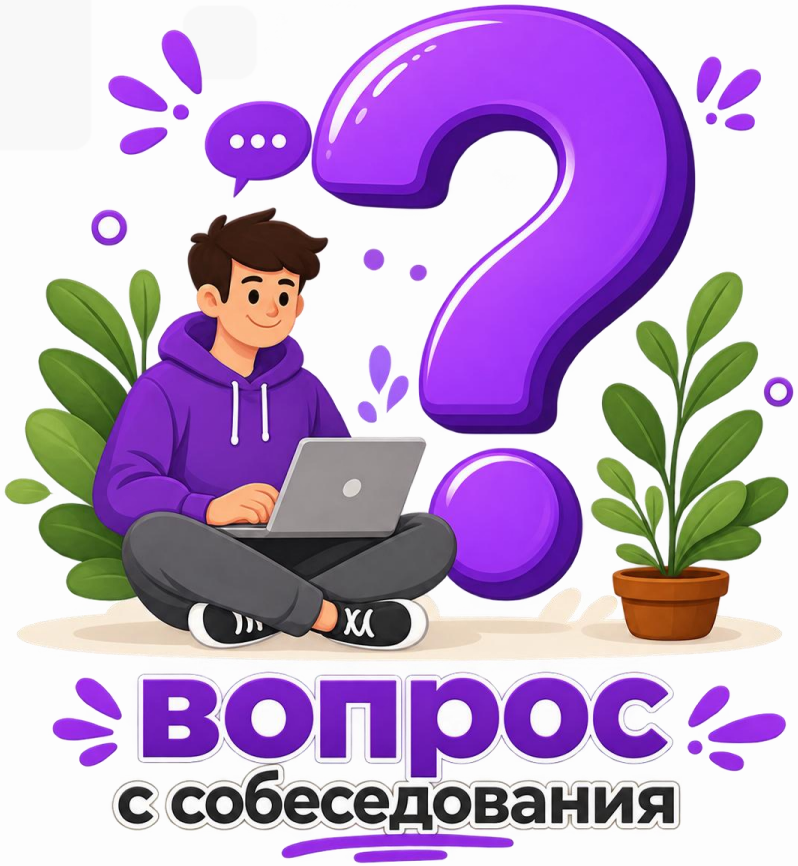
```java
@Test
public void testMyExceptionExample() {

    Assertions.assertThrows(MyException.class, () -> {
        exceptionsExample.myExceptionExample(200);
    });

    try {
        exceptionsExample.myExceptionExample(10);
        exceptionsExample.myExceptionExample(300);
    } catch (MyException e) {
        System.out.println("Exception: " + e);
    }
}
```



IThub



Расскажите про OutOfMemoryError

OutOfMemoryError выбрасывается, когда виртуальная машина Java не может создать (разместить) объект из-за нехватки памяти, а сборщик мусора не может высвободить достаточное её количество.

Область памяти, занимаемая java процессом, состоит из нескольких частей. Тип OutOfMemoryError зависит от того, в какой из них не хватило места:

java.lang.OutOfMemoryError: Java heap space: не хватает места в куче, а именно, в области памяти в которую помещаются объекты, создаваемые в приложении программно. Обычно проблема кроется в утечке памяти.

java.lang.OutOfMemoryError: GC overhead limit exceeded: связана с тем, что памяти осталось мало и сборщик мусора постоянно работает, пытаясь высвободить немного места. (можно отключить с помощью параметра -XX:-UseGCOverheadLimi)

java.lang.OutOfMemoryError: unable to create new native thread: выбрасывается, когда нет возможности создавать новые потоки.

Сцепленные исключения

- позволяют ассоциировать с одним исключением другое исключение, которое описывает причину первого исключения
- чтобы сделать возможными сцепленные исключения, в класс Throwable были добавлены два конструктора и два метода (также в Error, Exception и RuntimeException)

```
Throwable(Throwable causeExc)

Throwable(String msg, Throwable causeExc)
```

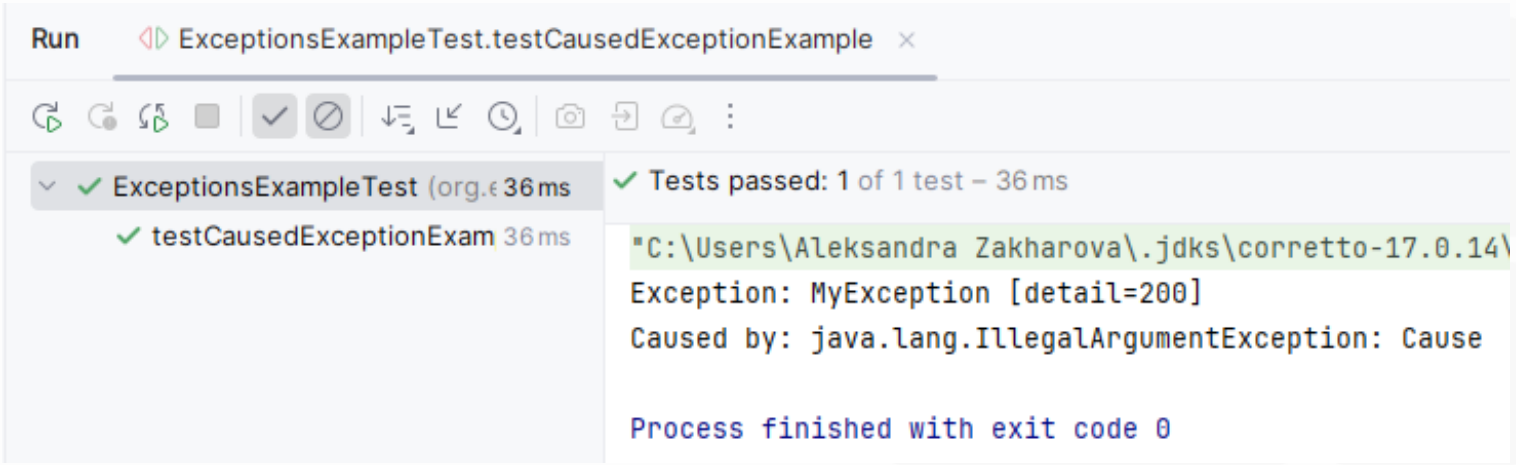
- Throwable поддерживает методы для сцепленных исключений
  - getCause() - возвращает исключение, лежащее в основе текущего исключения **или null** если его нет
  - initCause() - связывает causeExc с вызывающим исключением и возвращает ссылку на исключение

```java
public void causedExceptionExample() throws MyException{
    MyException myException = new MyException(200);
    myException.initCause(new IllegalArgumentException("Cause"));

    throw myException;
}
```

```java
@Test
public void testCausedExceptionExample() {
    try {
        exceptionsExample.causedExceptionExample();
    } catch (MyException exception) {
        System.out.println("Exception: " + exception);

        System.out.println("Caused by: " + exception.getCause()); // может быть null
    }
}
```



try с ресурсами

IThub

- автоматически закрывает используемые ресурсы после того, как выполнение блока try завершится. Можно избежать вручную закрытия ресурсов в блоке finally (обычно используется при работе с файлами).
- Под ресурсами подразумеваются сущности, реализующие интерфейс java.lang.Autocloseable.
- Блоки catch и явный finally выполняются уже после того, как закрываются ресурсы в неявном finally.

```
try(/*объявление ресурсов*/) {
    //...
} catch(Exception ex) {
    //...
} finally {
    //...
}
```

IThub

```
public void tryWithResourceExample() {
    try (InputStream in = new FileInputStream("file.txt")) {
        // считывание данных из файла
    } catch (IOException exception) {
        // обработка ошибок ввода/вывода
        // здесь in будет автоматически закрыт
        System.out.println("Exception: " + exception.getMessage());
    }

    String string = "This is test string";
    Reader reader = new StringReader(string);
    BufferedReader bufferedReader = new BufferedReader(reader);
    try (bufferedReader) {
        System.out.println(bufferedReader.readLine());
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Обработка ошибок

IThub

- используйте исключения вместо кодов ошибок
- начинайте с написания try-catch-finally (поможет определить, чего должен ожидать пользователь кода, что бы ни произошло в коде try)
- используйте непроверяемые исключения. Цена проверяемых исключений — нарушение принципа открытости/закрытости. Если инициировать проверяемое исключение из метода, а catch находится тремя уровнями выше, то это исключение должно быть объявлено в сигнатурах всех методов между данным методом и catch. Следовательно, изменение на низком уровне программного продукта приводит к изменениям сигнатур на многих более высоких уровнях
- каждое исключение, инициируемое в программе, должно содержать достаточно контекстной информации для определения источника и местонахождения ошибки
- не возвращайте null – выбросите исключение



Задача (КТ №3)

IThub

Создайте систему для управления библиотекой книг на языке Java

- **Создание базовых классов**:

Создайте абстрактный класс Book, который будет содержать основную информацию о книге: название, автор, год издания.
От класса Book унаследуйте два класса: FictionBook и NonFictionBook, представляющие художественные и научно-популярные книги соответственно.

- **Интерфейсы и полиморфизм**:

Создайте интерфейс Readable, который будет содержать метод read(). Реализуйте этот интерфейс в классе Book, чтобы любая книга могла быть прочитана.
Реализуйте метод read() в классах FictionBook и NonFictionBook, выводя соответствующее сообщение о типе книги.

- **Исключения**:

Создайте пользовательский класс исключения BookUnavailableException, который будет сигнализировать о том, что запрошенная книга недоступна.
В методах классов, отвечающих за выдачу и возврат книги, используйте блоки try-catch для обработки исключения BookUnavailableException.

- **Список книг с ArrayList**:

Создайте класс Library, который будет представлять библиотеку и хранить список книг с помощью ArrayList.
Реализуйте методы для добавления книги в библиотеку, выдачи и возврата книги.

- **Статические и вложенные классы**:

Создайте статический вложенный класс LibraryHelper в классе Library, который будет содержать статический метод для поиска книги по названию в списке.

- **Анонимные классы и перечисленные типы**:

Создайте перечисление Genre для определения жанра книги (например, FICTION, NON_FICTION).
В классе Library, реализуйте метод для поиска книг по жанру с использованием анонимных классов.

Протестируйте созданные классы и методы в main методе, создавая объекты книг, добавляя их в библиотеку, выдавая и возвращая, а также выполняя поиск книг по разным критериям.